AI Assisted Coding Ass-13.5

Code Refactoring – Improving Legacy Code with AI Suggestions

2403A52L05 – B50

E. Sai Nithin Ramanujan

Task-1:

```python
"""Task 1: Refactor duplicated rectangle calculations into reusable functions."""


def calculate_rectangle_metrics(length, width):
    """Return area and perimeter for a rectangle.

    Args:
        length: The rectangle length.
        width: The rectangle width.

    Returns:
        A tuple containing (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


def print_rectangle_metrics(length, width):
    """Print area and perimeter for a rectangle in the required format.

    Args:
        length: The rectangle length.
        width: The rectangle width.
    """
    area, perimeter = calculate_rectangle_metrics(length, width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)


rectangles = [(5, 10), (7, 12), (10, 15)]

input("Press Enter to display rectangle area and perimeter results... ")

for rectangle_length, rectangle_width in rectangles:
    print_rectangle_metrics(rectangle_length, rectangle_width)
```

Output:

```
PS D:\Collage\AI-AC> & "C:\Users\Sai Nithin\AppData\Local\Programs\Python\Python311\python.exe" d:/Collage/AI-AC/week-13/Task-1.py
Press Enter to display rectangle area and perimeter results...
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task-2:

```python
"""Task 2: Refactor inline tax/total calculations into reusable functions."""


def calculate_total(price):
    """Calculate total price including 18% tax.

    Args:
        price: Base price before tax.

    Returns:
        Final total price after adding 18% tax.
    """
    tax = price * 0.18
    return price + tax


def display_total(price):
    """Print the total price for a given base price.

    Args:
        price: Base price before tax.
    """
    total = calculate_total(price)
    print("Total Price:", total)


prices = [250, 500]

for item_price in prices:
    display_total(item_price)
```

Output:

```
=== Task-2 (PASS) ===
Total Price: 295.0
Total Price: 590.0
```

Task-3:

```python
"""Task 3: Refactor repeated grading conditionals into a class-based design."""


class GradeCalculator:
    """Calculate grades from marks using reusable grading rules."""

    def calculate_grade(self, marks):
        """Return a grade string for the given marks.

        Args:
            marks: Student marks (expected range: 0 to 100).

        Returns:
            A grade string based on the configured grading logic.
        """
        if 90 <= marks <= 100:
            return "Grade A"
        if marks >= 80:
            return "Grade B"
        if marks >= 70:
            return "Grade C"
        if marks >= 40:
            return "Grade D"
        if marks >= 0:
            return "Fail"
        return "Invalid marks"


calculator = GradeCalculator()
sample_marks = [85, 72, 95, 38]

for marks_value in sample_marks:
    print(calculator.calculate_grade(marks_value))
```

Output:

```
=== Task-3 (PASS) ===
Grade B
Grade C
Grade A
Fail
```

Task-4:

```python
"""Task 4: Refactor procedural square calculation into modular functions."""


def get_input():
    """Read an integer number from the user.

    Returns:
        The integer entered by the user.
    """
    return int(input("Enter number: "))


def calculate_square(num):
    """Calculate the square of a number.

    Args:
        num: Integer value to square.

    Returns:
        The square of the input number.
    """
    return num * num


def display_result(square):
    """Display the computed square value.

    Args:
        square: Computed square value.
    """
    print("Square:", square)


number = get_input()
square_value = calculate_square(number)
display_result(square_value)
```

Output:

```
=== Task-4 (PASS) ===
Enter number: Square: 81
```

Task-5

```python
"""Task 5: Refactor procedural salary logic into an OOP design."""


class EmployeeSalaryCalculator:
    """Calculate tax and net salary using encapsulated salary data."""

    def __init__(self, salary, tax_rate=0.2):
        """Initialize salary calculator values.

        Args:
            salary: Gross salary amount.
            tax_rate: Tax rate applied to salary. Defaults to 0.2 (20%).
        """
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self):
        """Return the tax amount based on salary and tax rate."""
        return self.salary * self.tax_rate

    def calculate_net_salary(self):
        """Return net salary after deducting tax."""
        return self.salary - self.calculate_tax()

    def display_net_salary(self):
        """Print the computed net salary."""
        print(self.calculate_net_salary())


employee_salary = EmployeeSalaryCalculator(50000)
employee_salary.display_net_salary()
```

Output:

```
=== Task-5 (PASS) ===
40000.0
```

Task-6

week-13 > Task-6.py > ...

```python
"""Task 6: Optimize username search using a set for O(1) average lookup."""


def has_access(username, allowed_users):
    """Check whether a username exists in the allowed users set.

    Args:
        username: Username entered by the user.
        allowed_users: Set of usernames with access.

    Returns:
        True if username is found, otherwise False.

    Complexity:
        Average-case membership check in a set is O(1),
        improving over O(n) linear search in a list.
    """
    return username in allowed_users


users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")

print("Access Granted" if has_access(name, users) else "Access Denied")

```

Output:

```
=== Task-6 (PASS) ===
Enter username: Access Granted
```

Task-7

```python
"""Library management module with reusable CRUD-style functions."""

library_db = {}


def add_book(title, author, isbn):
    """Add a book to the in-memory library database.

    Args:
        title: Book title.
        author: Book author name.
        isbn: Unique ISBN identifier used as key.

    Returns:
        True if the book is added successfully, False if ISBN already exists.
    """
    if isbn in library_db:
        return False

    library_db[isbn] = {"title": title, "author": author}
    return True


def remove_book(isbn):
    """Remove a book from the in-memory library database by ISBN.

    Args:
        isbn: Unique ISBN identifier of the book to remove.

    Returns:
        True if the book is removed, False if ISBN is not found.
    """
    if isbn not in library_db:
        return False

    del library_db[isbn]
    return True
```

```python
def search_book(isbn):
    """Search for a book in the in-memory library database.

    Args:
        isbn: Unique ISBN identifier of the book to search.

    Returns:
        A dictionary with book details when found, otherwise None.
    """
    return library_db.get(isbn)
```

Output:

index

library d:\college ai ac\week-13\library.py

Library management module with reusable CRUD-style functions.

Functions

add_book(title, author, isbn)
Add a book to the in-memory library database.

Args:
title: Book title.
author: Book author name.
isbn: Unique ISBN identifier used as key.

Returns:
True if the book is added successfully, False if ISBN already exists.

remove_book(isbn)
Remove a book from the in-memory library database by ISBN.

Args:
isbn: Unique ISBN identifier of the book to remove.

Returns:
True if the book is removed, False if ISBN is not found.

Task-8

```python
"""Task 8: Fibonacci generator refactored into reusable functions."""

def generate_fibonacci(n):
    """Generate the Fibonacci sequence up to n terms.
    Args:
        n: Number of terms to generate.
    Returns:
        A list containing the Fibonacci sequence up to n terms.
    Raises:
        ValueError: If n is less than 1.
    """
    if n < 1:
        raise ValueError("n must be a positive integer")
    if n == 1:
        return [0]
    sequence = [0, 1]
    while len(sequence) < n:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence
def generate_fibonacci_legacy_style(n):
    if n < 1:
        raise ValueError("n must be a positive integer")
    if n == 1:
        return [0]
    values = [0, 1]
    a = 0
    b = 1
    for _ in range(2, n):
        c = a + b
        values.append(c)
        a = b
        b = c
    return values
```

```python
def compare_refactored_vs_legacy(n):
    """Compare refactored and legacy-style Fibonacci outputs for n terms.

    Args:
        n: Number of terms.

    Returns:
        True if both outputs match, otherwise False.
    """
    return generate_fibonacci(n) == generate_fibonacci_legacy_style(n)


if __name__ == "__main__":
    limit = int(input("Enter limit: "))
    series = generate_fibonacci(limit)
    print("Fibonacci Series:", series)
    print("Matches legacy logic:", compare_refactored_vs_legacy(limit))
```

Output:

```
=== Task-8 (PASS) ===
Enter limit: Fibonacci Series: [0, 1, 1, 2, 3, 5]
Matches legacy logic: True
```

Task-9

```python
"""Task 9: Twin primes checker refactored into reusable functions."""


def is_prime(n):
    """Return True if n is a prime number, otherwise False.

    Args:
        n: Integer to test for primality.

    Returns:
        True when n is prime, otherwise False.
    """
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    divisor = 3
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor += 2
    return True


def is_twin_prime(p1, p2):
    """Return True if p1 and p2 are twin primes.

    Args:
        p1: First integer.
        p2: Second integer.

    Returns:
        True if both numbers are prime and differ by 2, otherwise False.
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
```

```python
def find_twin_primes(start, end):
    """Generate all twin-prime pairs in the inclusive range [start, end].

    Args:
        start: Range start value.
        end: Range end value.

    Returns:
        A list of tuples representing twin-prime pairs.
    """
    pairs = []
    for number in range(max(2, start), end - 1):
        if is_twin_prime(number, number + 2) and number + 2 <= end:
            pairs.append((number, number + 2))
    return pairs


if __name__ == "__main__":
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print("Twin Primes" if is_twin_prime(a, b) else "Not Twin Primes")

    start_value = int(input("Enter range start: "))
    end_value = int(input("Enter range end: "))
    print("Twin prime pairs:", find_twin_primes(start_value, end_value))
```

Output:

```
=== Task-9 (PASS) ===
Enter first number: Enter second number: Twin Primes
Enter range start: Enter range end: Twin prime pairs: [(3, 5), (5, 7), (11, 13), (17, 19)]
```

Task-10

```python
"""Task 10: Refactor Chinese Zodiac logic into a modular implementation."""
def get_zodiac(year):
    """Return the Chinese Zodiac sign for a given year.
    Args:
        year: Integer year value.
    Returns:
        The corresponding Chinese Zodiac sign as a string.
    """
    zodiac_by_remainder = [
        "Monkey",
        "Rooster",
        "Dog",
        "Pig",
        "Rat",
        "Ox",
        "Tiger",
        "Rabbit",
        "Dragon",
        "Snake",
        "Horse",
        "Goat",
    ]
    return zodiac_by_remainder[year % 12]
def read_year():
    """Read a year from user input.
    Returns:
        Year entered by the user as an integer.
    """
    return int(input("Enter a year: "))
def display_zodiac(sign):
    """Display the zodiac sign.
    Args:
        sign: Zodiac sign string.
    """
    print(sign)
if __name__ == "__main__":
    year_value = read_year()
    zodiac_sign = get_zodiac(year_value)
    display_zodiac(zodiac_sign)
```

Output:

```
=== Task-10 (PASS) ===
Enter a year: Dragon
```

Task-11

```python
"""Task 11: Refactor Harshad checker into clean, reusable functions."""

def is_harshad(number):
    """Return whether a number is a Harshad (Niven) number.
    A Harshad number is divisible by the sum of its digits.
    Args:
        number: Integer value to check.
    Returns:
        True if the number is Harshad, otherwise False.
        Returns False for zero and negative values.
    """
    if number <= 0:
        return False

    digit_sum = sum(int(digit) for digit in str(number))
    if digit_sum == 0:
        return False
    return number % digit_sum == 0


def read_number():
    """Read an integer from user input."""
    return int(input("Enter a number: "))


def display_result(result):
    """Display the Harshad check result.

    Args:
        result: Boolean result from is_harshad.
    """
    print(result)


if __name__ == "__main__":
    user_number = read_number()
    display_result(is_harshad(user_number))
```

Output:

```
=== Task-11 (PASS) ===
Enter a number: True
```

Task-12

```python
"""Task 12: Efficient trailing-zeros calculator for factorial values."""

def count_trailing_zeros(n):
    """Return the number of trailing zeros in n! without computing n! directly.
    Uses the optimized mathematical approach of counting factors of 5 in n!.
    Args:
        n: Non-negative integer.
    Returns:
        Number of trailing zeros in n!.
    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")
    count = 0
    divisor = 5
    while divisor <= n:
        count += n // divisor
        divisor *= 5
    return count
def read_number():
    """Read an integer from user input."""
    return int(input("Enter a number: "))
def display_result(n, zeros):
    """Display trailing-zero result in expected format.
    Args:
        n: Input number.
        zeros: Computed trailing zero count for n!.
    """
    print(f"Trailing zeros: {zeros}")


if __name__ == "__main__":
    try:
        number = read_number()
        result = count_trailing_zeros(number)
        display_result(number, result)
    except ValueError as error:
        print(error)
```

Output:

```
=== Task-12 (PASS) ===
Enter a number: Trailing zeros: 2
```

Task-13

```python
"""Task 13: Collatz sequence generator with validation."""


def generate_collatz_sequence(n):
    """Generate the Collatz sequence starting from n until reaching 1.

    Args:
        n: Starting positive integer.

    Returns:
        A list containing the Collatz sequence from n to 1.

    Raises:
        ValueError: If n is less than 1.
    """
    if n < 1:
        raise ValueError("n must be a positive integer")

    sequence = [n]
    while n != 1:
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
        sequence.append(n)
    return sequence


if __name__ == "__main__":
    start = int(input("Enter a positive integer: "))
    print(generate_collatz_sequence(start))
```

Output:

```
=== Task-13 (PASS) ===
Enter a positive integer: [6, 3, 10, 5, 16, 8, 4, 2, 1]
```

Task-14

```python
"""Task 14: Lucas sequence generator with reusable function."""


def generate_lucas_sequence(n):
    """Generate the Lucas sequence up to n terms.

    Lucas starts with 2, 1 and follows:
        L(n) = L(n-1) + L(n-2)

    Args:
        n: Number of terms to generate.

    Returns:
        A list containing n Lucas terms.

    Raises:
        ValueError: If n is less than 1.
    """
    if n < 1:
        raise ValueError("n must be a positive integer")
    if n == 1:
        return [2]

    sequence = [2, 1]
    while len(sequence) < n:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence


if __name__ == "__main__":
    terms = int(input("Enter number of terms: "))
    print(generate_lucas_sequence(terms))
```

Output:

```
=== Task-14 (PASS) ===
Enter number of terms: [2, 1, 3, 4, 7]
```

Task-15

```python
"""Task 15: Count vowels and consonants in a string."""


def count_vowels_and_consonants(text):
    """Count vowels and consonants in the given text.

    Only alphabetic characters are counted. Non-letters are ignored.

    Args:
        text: Input string.

    Returns:
        A tuple (vowel_count, consonant_count).
    """
    vowels = set("aeiouAEIOU")
    vowel_count = 0
    consonant_count = 0

    for char in text:
        if not char.isalpha():
            continue
        if char in vowels:
            vowel_count += 1
        else:
            consonant_count += 1

    return vowel_count, consonant_count


if __name__ == "__main__":
    user_text = input("Enter text: ")
    vowels, consonants = count_vowels_and_consonants(user_text)
    print(f"Vowels: {vowels}, Consonants: {consonants}")
```

Output:

```
=== Task-15 (PASS) ===
Enter text: Vowels: 2, Consonants: 3
```

Task-16

```python
"""Task 16: Refactor to remove unnecessary global variables."""


def calculate_interest(amount, rate):
    """Calculate simple interest using provided parameters.

    Args:
        amount: Principal amount.
        rate: Interest rate.

    Returns:
        Calculated interest value.
    """
    return amount * rate


if __name__ == "__main__":
    config = {"rate": 0.1}
    print(calculate_interest(1000, config["rate"]))
```

Output:

```
=== Task-16 (PASS) ===
100.0
```

Task-17

week-13 > Task-17.py > ...

```python
"""Task 17: Refactor deeply nested conditionals into flat logic."""


def get_score_feedback(score):
    """Return feedback message based on score thresholds.

    Args:
        score: Numeric score value.

    Returns:
        Feedback message string.
    """
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"


if __name__ == "__main__":
    score = 78
    print(get_score_feedback(score))

```

Output:

```
=== Task-17 (PASS) ===
Very Good
```

Task-18

week-13 > Task-18.py > ...

```python
"""Task 18: Refactor repeated file handling using reusable functions."""


def read_file_contents(file_path):
    """Read and return file contents from the given path.

    Args:
        file_path: Path to the file.

    Returns:
        File contents as a string.
    """
    with open(file_path, "r", encoding="utf-8") as file:
        return file.read()


def print_file_contents(file_path):
    """Print contents of a file.

    Args:
        file_path: Path to the file.
    """
    print(read_file_contents(file_path))


if __name__ == "__main__":
    for path in ("data1.txt", "data2.txt"):
        print_file_contents(path)

```

Output:

```
=== Task-18 (PASS) ===
This is sample content from data1.txt.

This is sample content from data2.txt.
```

Task-19

week-13 > Task-19.py > has_access

```python
"""Task 19: Optimize user lookup with a set-based membership check."""


def has_access(username, allowed_users):
    """Return whether a username exists in allowed users.

    Args:
        username: Input username.
        allowed_users: Set of authorized usernames.

    Returns:
        True if username exists, otherwise False.

    Complexity:
        Average O(1) lookup using a set instead of O(n) list scan.
    """
    return username in allowed_users


if __name__ == "__main__":
    users = {"admin", "guest", "editor", "viewer"}
    name = input("Enter username: ")
    print("Access Granted" if has_access(name, users) else "Access Denied")

```